

Trabajo Práctico N° 2

Paradigmas de Programación

Integrantes:

Vega Lautaro 44837673

Vargas Tomás 44822900

Arenas Velasco Artin 44510867

Rodriguez Gonzalo Ezequiel 46573724

Churquina Diego Sebastian 40394243

Universidad Nacional de La Matanza

Comisión: 01-5900 Turno: Viernes Noche

Docentes: Fabian Lagorio, Victor Fernandez

2 de julio de 2026

Índice

Caratula.....	1
Índice.....	2
1. Introducción.....	3
2. Analisis del problema.....	3
3. Diseño de la solución.....	3
4. Patrones de diseño utilizados.....	4
5. Conclusion.....	5

1. Introducción

En este trabajo práctico se desarrolló un sistema que simula una batalla entre magos y mortífagos, aplicando los conceptos vistos durante la cursada. Para su desarrollo se utilizaron clases, herencia, polimorfismo, interfaces y otros recursos propios de este paradigma, con el objetivo de construir un programa organizado y fácil de ampliar.

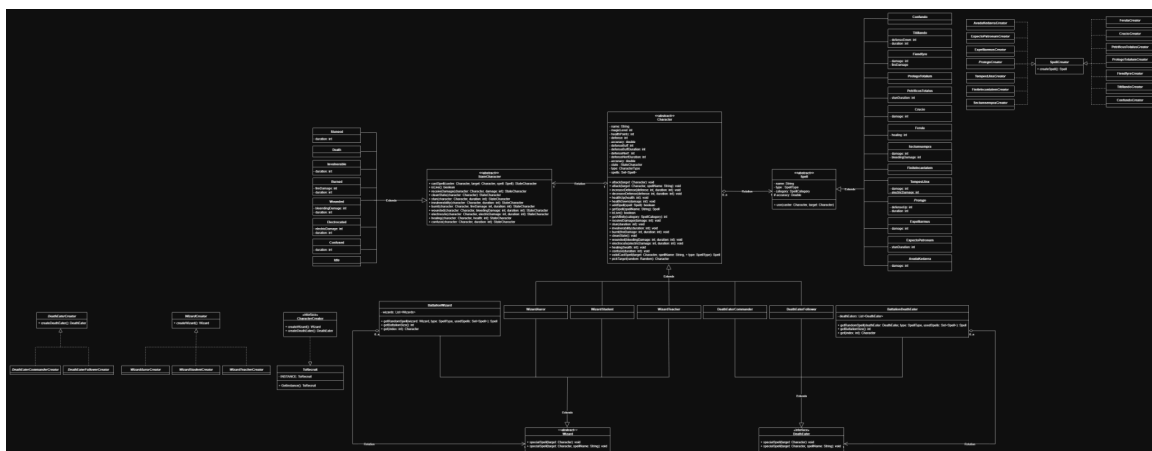
En este informe se describen las principales decisiones tomadas durante el desarrollo del proyecto, el diseño de la solución implementada y los patrones de diseño utilizados, acompañados por el diagrama de clases correspondiente.

El programa para su correcta ejecución requiere java 22 o superior.

2. Analisis del problema

El problema planteado consiste en desarrollar un sistema capaz de simular una batalla entre dos facciones enfrentadas: magos y mortífagos. Cada personaje posee características propias, habilidades y hechizos que influyen en el desarrollo del combate. El sistema debe representar estas diferencias de manera flexible, permitiendo que el comportamiento de cada personaje dependa de su tipo sin recurrir a estructuras condicionales que dificulten su mantenimiento y escalabilidad.

3. Diseño de la solución



4. Patrones de diseño utilizados

Para centralizar y ocultar la creación de magos, mortífagos y hechizos se implementó el patrón de diseño **Factory Method**, evitando que el resto del sistema conozca los detalles de construcción de cada objeto. De esta manera, la creación de personajes y hechizos queda desacoplada de su utilización, facilitando el mantenimiento y la incorporación de nuevas variantes.

Además, se empleó el patrón **Abstract Factory** para encapsular la creación de las distintas facciones. A través de una fábricas concretas es posible generar tipo específico de personaje de forma aleatoria, permitiendo crear batallones variados sin exponer la lógica de instanciación al resto de la aplicación.

La clase **ToRecruit**, encargada de actuar como la fábrica principal del sistema, implementa el patrón de diseño **Singleton**. Este patrón garantiza que exista una única instancia de la fábrica durante toda la ejecución del programa, evitando la creación innecesaria de múltiples objetos que cumplirían exactamente la misma función. Gracias a esta implementación, cualquier componente del sistema que necesite crear un personaje puede acceder a la misma instancia mediante el método estático **GetInstance()**, sin necesidad de invocar repetidamente el constructor de la clase. Esto centraliza la responsabilidad de la creación de personajes y simplifica el acceso a la fábrica desde cualquier punto de la aplicación.

Para gestionar los distintos estados que puede presentar un personaje durante la batalla se implementó el patrón de diseño **State**. Este patrón permite que el comportamiento de un personaje varíe dinámicamente según su estado actual, evitando el uso de múltiples estructuras condicionales (if o switch) para determinar qué acciones puede realizar.

Cada estado del personaje se representa mediante una clase que hereda de **StateCharacter** y redefine únicamente los métodos cuyo comportamiento difiere del estado normal. De esta manera, acciones como lanzar un hechizo, recibir daño o aplicar un nuevo efecto dependen del objeto estado asociado al personaje y no de comprobaciones explícitas sobre su condición.

Para modelar los distintos hechizos se implementó el patrón de diseño **Strategy**. Cada hechizo se representa mediante una clase que hereda de **Spell** e implementa el método **use**, encapsulando el comportamiento específico de dicho hechizo. De esta manera, la lógica de ejecución de cada uno permanece independiente del resto del sistema.

Gracias a este diseño, los personajes únicamente interactúan con la abstracción **Spell**, sin necesidad de conocer la implementación concreta del hechizo que están utilizando. Esto evita el uso de estructuras condicionales (if o switch) para determinar qué efecto debe aplicarse, ya que el comportamiento correcto surge mediante polimorfismo. La incorporación de un nuevo hechizo resulta sencilla, ya que únicamente es necesario crear una nueva subclase de **Spell** e implementar el método **use** con el efecto correspondiente.

Para representar los batallones de magos y mortífagos se implementó el patrón de diseño **Composite**. Este patrón permite tratar de forma uniforme tanto a un personaje individual como a un conjunto de personajes, ya que los batallones heredan de la misma jerarquía que sus integrantes y exponen la misma interfaz pública.

Cada batallón actúa como un objeto compuesto que contiene una colección de personajes de su misma facción. Cuando se invoca una operación sobre el batallón, como atacar, apoyar a un aliado, recibir daño o aplicar un efecto de estado, este delega la ejecución a cada uno de sus integrantes.

5. Conclusion

El desarrollo del trabajo práctico permitió implementar una solución que cumple con los requerimientos planteados en la consigna. La organización de las clases, la jerarquía de personajes y la implementación de los distintos hechizos permitieron construir una estructura clara y ordenada para representar la lógica de las batallas.

En conjunto, se obtuvo una aplicación funcional en la que cada componente cumple una responsabilidad específica dentro del sistema, permitiendo mantener una estructura organizada y un funcionamiento coherente entre los distintos elementos que participan en la simulación.

Asimismo, la incorporación de patrones de diseño como **Factory Method**, **Abstract Factory**, **Singleton**, **Strategy**, **State** y **Composite** permitió construir una arquitectura flexible y desacoplada. Gracias a estas decisiones de diseño, el sistema puede ampliarse con nuevos personajes, hechizos, estados o mecánicas de combate sin requerir modificaciones importantes en las clases existentes, favoreciendo el cumplimiento del principio de abierto/cerrado.